

# Parking Enforcement Intelligence

A Predictive Hotspot Detection & Enforcement Prioritisation  
System for Bengaluru

*Technical Documentation, Gridlock Hackathon 2.0, Round 2*

**Team: Solaris**

Theme: Parking-Induced Congestion

June 20, 2026

# Contents

|           |                                                                                             |           |
|-----------|---------------------------------------------------------------------------------------------|-----------|
| <b>1</b>  | <b>Executive Summary</b>                                                                    | <b>3</b>  |
| <b>2</b>  | <b>Problem Statement &amp; Roadmap</b>                                                      | <b>3</b>  |
| 2.1       | Original Problem Brief                                                                      | 3         |
| 2.2       | Decomposing the Problem into a Sequenced Roadmap                                            | 3         |
| <b>3</b>  | <b>Foundational Assumptions &amp; Constraints</b>                                           | <b>4</b>  |
| 3.1       | “Caught” is treated as a proxy for “Occurred”                                               | 4         |
| 3.2       | Profiling, not causal inference                                                             | 5         |
| 3.3       | Workforce and patrol planning is an Operations Research problem, not a Machine Learning one | 5         |
| 3.4       | No external stakeholder dependency on causal claims                                         | 5         |
| 3.5       | Dataset scope and time window                                                               | 5         |
| <b>4</b>  | <b>System Architecture Overview</b>                                                         | <b>5</b>  |
| 4.1       | High-Level Component Map                                                                    | 6         |
| <b>5</b>  | <b>Historic Hotspot Detection</b>                                                           | <b>7</b>  |
| 5.1       | Spatial Clustering Methodology                                                              | 7         |
| 5.2       | PCU-Weighted Impact, Not Raw Violation Count                                                | 7         |
| <b>6</b>  | <b>Hotspot Classification: Duration &amp; Burstiness</b>                                    | <b>7</b>  |
| <b>7</b>  | <b>Predictive Engine: Calendar-Similar K-Nearest-Neighbours</b>                             | <b>8</b>  |
| 7.1       | Design Rationale                                                                            | 8         |
| 7.2       | Calendar Feature Engineering                                                                | 8         |
| 7.3       | Distance Metric & Feature Weighting                                                         | 9         |
| 7.4       | K-Nearest-Neighbours Mechanics                                                              | 9         |
| 7.5       | Iterative Refinement                                                                        | 9         |
| <b>8</b>  | <b>Hotspot Profiling</b>                                                                    | <b>10</b> |
| <b>9</b>  | <b>Decision Engine: Strategy Selection &amp; Resource Allocation</b>                        | <b>10</b> |
| 9.1       | Rule-Based Strategy Selection                                                               | 10        |
| 9.2       | Resource Allocation as a Linear Programming Problem                                         | 10        |
| <b>10</b> | <b>Live Data Pipeline &amp; Feedback Mechanism</b>                                          | <b>11</b> |
| 10.1      | Live Ingestion (Proof of Concept)                                                           | 11        |
| 10.2      | Feedback Mechanism: Current Scope and Future Direction                                      | 11        |
| <b>11</b> | <b>Model Evaluation Methodology</b>                                                         | <b>11</b> |
| 11.1      | Chronological Train / Validation / Test Split                                               | 11        |
| 11.2      | Walk-Forward (Rolling-Origin) Validation                                                    | 12        |
| 11.3      | Ground Truth Definition                                                                     | 12        |
| 11.4      | Metrics                                                                                     | 12        |
| 11.5      | Confidence Threshold Selection                                                              | 13        |
| 11.6      | Validation Results                                                                          | 13        |

---

|                                                   |           |
|---------------------------------------------------|-----------|
| <b>12 Dashboard &amp; Frontend Implementation</b> | <b>14</b> |
| 12.1 Layout . . . . .                             | 14        |
| <b>13 Future Scope</b>                            | <b>15</b> |
| <b>Appendix A: Technology Stack Summary</b>       | <b>16</b> |

## 1 | Executive Summary

This document describes the technical design, implementation, and evaluation methodology behind our submission for the Gridlock Hackathon 2.0, Round 2, under the theme “*Poor Visibility on Parking-Induced Congestion*.” The system ingests historical parking-violation enforcement records for Bengaluru and produces three connected capabilities: (1) historic hotspot identification using density-based spatial clustering, (2) a calendar-similarity  $k$ -Nearest-Neighbours model that predicts which zones are likely to be violation hotspots on a future date, and (3) a rule-based decision-support layer that maps each hotspot’s profile to internationally predated parking-management strategies and a resource-allocation recommendation.

The submission is presented as a working prototype, not a finished production system. Several components, including workforce scheduling, patrol routing, and the live-feedback mechanism, are deliberately implemented as simplified, hardcoded stand-ins for now, since the core analytical pipeline (hotspot detection, classification, and prediction) was prioritised first. This document states clearly what is fully implemented, what is a placeholder, and what remains open as a modelling or engineering decision. We believe this level of detail is itself a meaningful part of demonstrating engineering maturity within a hackathon timeline.

Section 2 covers the original problem framing and how we broke it down into a sequenced roadmap. Section 3 documents the foundational assumptions we made and why. Sections 4 to 10 describe the system architecture and each pipeline stage in the order data flows through it. Section 11 covers our evaluation methodology in detail, including the train/validation/test discipline we adopted and the metrics we used to judge the predictive model honestly. Section 12 documents the dashboard implementation, illustrated with screenshots of the working system, and Section 13 covers future scope.

## 2 | Problem Statement & Roadmap

### 2.1 Original Problem Brief

The challenge brief (“*Poor Visibility on Parking-Induced Congestion*”) identified that on-street illegal and spillover parking near commercial areas, metro stations, and event venues chokes carriageways and intersections across Bengaluru. Enforcement today is patrol-based and reactive, with no heatmap of parking violations against congestion impact and no principled way to prioritise where enforcement effort should go. The brief’s problem-statement direction asked: “*How can AI-driven parking intelligence detect illegal parking hotspots and quantify their impact on traffic flow to enable targeted enforcement?*” A historical violations dataset (around 298,000 records, November 2023 to April 2024, covering 55 police-station jurisdictions across Bengaluru) was provided as the starting point.

### 2.2 Decomposing the Problem into a Sequenced Roadmap

Early in the project we broke the end-to-end system down into eight sequential capabilities, labelled a through h below, recognising that later stages depend on outputs from earlier ones, and that not all stages carry equal risk or are equally well-suited to a hackathon timeline. This breakdown shaped our entire build order and is referenced throughout this document.

| Stage | Capability                                     | Status in this submission                                                                                          |
|-------|------------------------------------------------|--------------------------------------------------------------------------------------------------------------------|
| a     | Identify historic hotspots                     | Implemented: grid-based density clustering                                                                         |
| b     | Classify hotspots by time duration / intensity | Implemented: duration and burstiness typology                                                                      |
| c     | Predict current/future day's hotspots          | Implemented: calendar-similarity KNN, walk-forward validated                                                       |
| d     | Identify cause / profile of each hotspot       | Implemented: descriptive profiling (vehicle, landmark, time)                                                       |
| e     | Schedule workforce by intensity                | Partial. Linear-programming allocator built but not fully wired into the live path; simplified heuristic used live |
| f     | Define patrol routes & timing                  | Out of scope for this round. Identified as an Operations Research problem for future work                          |
| g     | Live data-stream ingestion pipeline            | Partial. Simulated ingestion endpoint built as a proof of concept                                                  |
| h     | Feedback mechanism                             | Partial. Simplified zone-level mitigation flag implemented; full vehicle-impact reweighting scoped as future work  |

Stages (a) through (d) form the analytical core of the submission and received the majority of our engineering and validation effort. Stages (e) through (h) were deliberately scoped down once it became clear they depend on operational constraints, such as staffing levels, shift rules, and real deployment feedback, that cannot be derived from a five-month historical violations dataset alone. We treat this scoping decision as a feature of disciplined project management rather than a shortfall: building a confident, well-evaluated prediction engine was judged more valuable within the time available than building a shallow version of all eight stages.

### 3 | Foundational Assumptions & Constraints

Every analytical system rests on assumptions about its data and its scope. We chose to state ours plainly rather than leave them implicit, since an unstated assumption is far more likely to be mistaken for a validated fact by someone reading the output later.

#### 3.1 “Caught” is treated as a proxy for “Occurred”

The violations dataset reflects where and when enforcement recorded a violation. It does not, and cannot, directly measure where illegal parking actually occurs most. A zone with more cameras or more active patrols will show more violations even if underlying driver behaviour is similar elsewhere. We do not have an independent ground-truth source (for example, manual audits or a second sensor type) to correct for this enforcement-intensity bias at this stage, so for

the current phase of the project we treat “caught” and “occurred” as equivalent. This assumption is carried through every hotspot, prediction, and profile produced by the system and should be understood as a limitation of the current dataset, not of the modelling approach.

### 3.2 Profiling, not causal inference

Stage (d) of our roadmap characterises each hotspot by its dominant vehicle type, nearby landmark or junction context, peak time of day, and day type. This is descriptive profiling: it answers “what characterises this hotspot,” not causal inference, which would answer “why does this hotspot occur” or “what intervention would reduce it.” We use the term *hotspot profiling* on purpose throughout this document and the codebase, to avoid implying a causal claim the underlying data cannot support.

### 3.3 Workforce and patrol planning is an Operations Research problem, not a Machine Learning one

Converting a ranked, classified hotspot list into actual headcount, shift assignments, and patrol routes depends on variables the violations dataset does not contain at all: enforcement-action duration, shift length and break rules, inter-zone travel time, and existing staffing constraints. We scoped stages (e) and (f) as a downstream optimisation layer that consumes our model’s output as one input among several real-world operational constraints, rather than attempting to derive staffing decisions from historical violation data alone.

### 3.4 No external stakeholder dependency on causal claims

This submission is built for a hackathon evaluation context. There is no operational deployment currently consuming this system’s output to make real enforcement decisions. This affects how carefully we need to treat the descriptive-versus-causal distinction above: it is an important documentation and terminology discipline now, and would become a hard requirement before any real-world deployment.

### 3.5 Dataset scope and time window

The provided dataset spans roughly five months (November 2023 to April 2024), which limits how much can be learned about rare, calendar-driven effects such as major festivals, since most festivals appear only once or twice in this window. We treat this as a known limitation of the prediction model’s calendar-similarity features (Section 7) rather than a flaw in the modelling approach itself; the approach is designed to generalise as more historical data accumulates.

## 4 | System Architecture Overview

The system is organised around three distinct, clearly separated operating modes. Keeping these conceptually and operationally separate was a deliberate architectural decision, made after we noticed that several early bugs came from one mode’s logic unintentionally affecting another.

| Mode           |            | Purpose                                                                              | Data Window                                                                       | Trigger                                                                        |
|----------------|------------|--------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------|--------------------------------------------------------------------------------|
| Live Dashboard |            | Show the current real-world enforcement situation                                    | Rolling, most recent 7 days of actual data                                        | Default view, always on                                                        |
| Predict row    | Tomorrow   | Show the forecast for the next day                                                   | All available history up to today, no fixed window                                | User-controlled toggle; stays on until switched off                            |
| Offline        | Evaluation | Tune $K$ , feature weights, and confidence threshold; report model accuracy honestly | Walk-forward over historical data (chronological train / validation / test split) | Run by the development team only, never by an end user in the live application |

A core architectural rule we enforce is that the offline evaluation mode's tuning logic must never influence the live system's runtime behaviour. The evaluation mode exists to choose a threshold, a  $K$  value, and a feature-weight vector once, offline; the live prediction engine then simply applies those frozen, pre-chosen values against whatever history is actually available at the moment a prediction is requested.

## 4.1 High-Level Component Map

- **Data Processing Layer:** cleans raw violation records, parses multi-value violation-type tags, converts timestamps to IST, computes PCU-weighted impact per vehicle type.
- **Hotspot Detection Layer:** grid-based spatial clustering to identify physical hotspot zones independent of police-jurisdiction boundaries.
- **Classification Layer:** labels each hotspot by temporal duration and burstiness, informing patrol-response type.
- **Predictive Engine:** calendar-similarity  $k$ -NN model forecasting next-day hotspot likelihood and confidence per zone.
- **Decision Engine:** rule-based strategy selector mapping hotspot profile and classification to internationally precedented parking-management interventions, plus a linear-programming-based resource allocator.
- **Server / API Layer:** Python backend serving filtered live data, zone scenarios, and predictions to the frontend.
- **Dashboard Frontend:** interactive map (heatmap, ranked hotspots, station markers), time-pattern charts, violation/vehicle breakdowns, and a zone leaderboard, built with Leaflet and Chart.js.

## 5 | Historic Hotspot Detection

### 5.1 Spatial Clustering Methodology

Hotspots are detected using a grid-based density clustering approach: every violation point is snapped to a 100-metre grid cell, cells with at least a minimum violation count are kept, and adjacent dense cells are merged via 8-neighbour connected components into a single hotspot zone. This is, in effect, a memory-efficient, deterministic stand-in for DBSCAN. It produces results equivalent to a haversine-metric DBSCAN at this spatial resolution while avoiding the memory cost of building a ball-tree over the full dataset of around 298,000 points, which matters on constrained hardware. The result is hundreds of hotspot zones covering the large majority of all logged violations, independent of administrative police-station boundaries.

### 5.2 PCU-Weighted Impact, Not Raw Violation Count

An early version of this system ranked hotspots by raw violation count. We moved away from this in favour of a Passenger Car Unit (PCU) weighted impact score, since the original problem brief specifically asks about congestion impact, and a parked heavy goods vehicle occupies and obstructs road space very differently from a parked scooter. A raw count treats them identically. PCU weights are drawn from Indian Highway Capacity Manual conventions and applied per vehicle type before aggregating violations into a zone's impact score.

| Vehicle Category                          | PCU Weight  | Rationale                                                    |
|-------------------------------------------|-------------|--------------------------------------------------------------|
| Two-wheelers (scooter, motorcycle, moped) | 0.5         | Minimal road-space obstruction                               |
| Cars, jeeps, vans                         | 1.0         | Baseline reference unit                                      |
| Autos (passenger / goods)                 | 0.75 to 1.0 | Intermediate footprint                                       |
| Light commercial (LGV, tempo, maxi-cab)   | 1.5         | Larger footprint, lower frequency                            |
| Heavy vehicles (buses, lorries, HGV)      | 3.0 to 4.5  | Severe carriageway obstruction, highest enforcement priority |

## 6 | Hotspot Classification: Duration & Burstiness

Stage (b) of our roadmap classifies each detected hotspot along two operationally meaningful axes rather than relying on a single severity score:

- **Duration:** whether a hotspot's violations are concentrated in a short, fixed daily window (for example, a 40-minute spike around 2 PM) or spread across a long, sustained active period.
- **Burstiness:** whether activity within the active window forms a single sharp peak or multiple sporadic peaks.

These two axes were chosen specifically because they map directly onto a distinct patrol-

deployment pattern, which is the actual downstream decision this classification needs to support. A short-duration, single-peak hotspot calls for a single officer posted at a fixed time and place; a long-duration, multi-peak hotspot calls for a roaming beat or an extended shift. In the live dashboard, each hotspot is labelled with its classification directly (for example, **CHRONIC** for a long-duration, sustained hotspot, or **PEAK-DRIVEN** for one concentrated around a specific time window), visible on both the Zone Leaderboard and the per-zone profile panel on the map. We treat the typology as secondary to algorithm choice: simple peak detection or lightweight clustering over each hotspot's 24-hour violation profile is sufficient. The design effort went into choosing axes that are operationally actionable rather than into algorithmic sophistication.

## 7 | Predictive Engine: Calendar-Similar K-Nearest-Neighbours

### 7.1 Design Rationale

Stage (c) asks the system to predict which zones are likely to be hotspots on a given future day. We defined the prediction target narrowly. For each known hotspot zone, the goal is to estimate the probability that tomorrow's PCU-weighted violation count will meet or exceed that zone's own historical median, conditioned on calendar similarity to other historical days. This was chosen over a raw count-regression target because the system's actual downstream use is prioritisation, that is, "which zones need attention tomorrow," not an exact volume forecast.

### 7.2 Calendar Feature Engineering

Each historical (and future) date is represented as a feature vector capturing calendar context likely to shift parking and traffic behaviour:

- `is_festival`: whether the date is a religiously or culturally significant day in Karnataka, major or minor, holiday or not
- `is_holiday`: whether the date is an official, gazetted Karnataka government holiday
- `is_monday` / `is_tues_to_fri` / `is_sat_sun`: weekday-position indicators, refined from a single numeric day-of-week feature after recognising that treating day-of-week as an ordinal number implies false adjacency relationships (for example, that Wednesday sits meaningfully "between" Monday and Friday) that do not reflect real violation-pattern similarity
- `is_second_saturday`: a Karnataka-specific recurring bank and post-office holiday pattern

A dedicated reference module (`karnataka_calendar.py`) maintains explicit, sourced arrays of Karnataka holidays and festivals for the dataset's time window, including minor festivals that are culturally significant but not official holidays (for example, Varamahalakshmi Vratam, Karva Chauth), since these plausibly still shift shopping and parking behaviour even without an office closure. This replaced an earlier hardcoded, six-entry placeholder festival list used during initial development.

### 7.3 Distance Metric & Feature Weighting

Calendar similarity between two dates is computed as a weighted Euclidean distance over the feature vector above. We use a weighted rather than unweighted distance because the calendar features are not equally predictive of violation behaviour. Whether a date is a major festival plausibly reshapes the entire violation pattern, while an ordinary weekday-to-weekday distinction is a much smaller effect, and an unweighted metric cannot express that difference.

Feature weights were derived empirically using four independent methods: Random Forest feature importance, Linear Regression coefficient magnitude, Mutual Information, and Pearson Correlation, each computed against each candidate feature's relationship to daily violation impact. These methods do not always agree (notably, Random Forest, Mutual Information, and Correlation all rank weekday-position features highly, while Linear Regression ranks holiday status more highly), which we read as evidence of a non-linear relationship between weekday position and violation impact that a purely linear weighting scheme cannot fully capture. Since our distance metric is itself linear (Euclidean), we prioritised Correlation- and Linear-Regression-derived weights as the more methodologically consistent choice for this specific use, while still computing all four for comparison and as a check on robustness.

### 7.4 K-Nearest-Neighbours Mechanics

For a target date, the model finds the  $K$  historical dates with the smallest weighted calendar distance, with a secondary recency bonus that favours more recent matches among otherwise similarly distant candidates. For each zone independently, the  $K$  neighbour dates' historical PCU-impact values are combined into (a) a distance-weighted mean as the point estimate of predicted impact, and (b) a confidence score: the fraction of the  $K$  neighbour days on which that zone's impact met or exceeded its own historical median, which we refer to in evaluation as the *weighted neighbour agreement* (see Section 11 for why we avoid the word "probability" for this score). We settled on  $K = 20$  (roughly 15 to 17 percent of the candidate pool at the dataset's current size) as a balance between smoothing out single-day noise and staying genuinely local to the target date's calendar profile.

We considered and rejected predicting a fixed number of hotspots per day. Instead, a confidence threshold determines which zones are reported as predicted hotspots on a given day, allowing the predicted hotspot count to vary naturally with how calendar-eventful the day actually is, rather than forcing an artificial fixed list size.

### 7.5 Iterative Refinement

The predictive engine went through several rounds of refinement as we found gaps between an initial working version and a methodologically sound one:

1. Initial version: unweighted mean of  $K$  neighbours, hardcoded mock festival list, raw violation count as the predicted quantity.
2. Refined to a distance-weighted mean, consistent with the principle that a near-exact calendar match should count more than the  $K$ -th closest, weaker match.
3. Replaced raw count with PCU-weighted impact as the predicted quantity, for consistency with the rest of the system's severity currency (Section 5).

4. Replaced the mock festival list with a sourced, documented Karnataka holiday and festival reference module.
5. Expanded the calendar feature set from three features (day-of-week, is-festival, is-second-Saturday) to six, splitting day-of-week into weekday-position indicators after recognising the ordinality problem described above.

## 8 | Hotspot Profiling

Stage (d) characterises each hotspot using the dataset attributes available at that location: the dominant (modal) vehicle type, the nearest named junction or landmark (or “street segment” where no named junction is present), and the peak hour of violation activity. As stated in Section 3, this is presented and labelled throughout as descriptive profiling, not causal attribution. The data can support a characterisation such as “this hotspot is dominated by two-wheelers near a school during pickup hours,” but it cannot support a causal claim about what specific intervention would change that pattern.

## 9 | Decision Engine: Strategy Selection & Resource Allocation

### 9.1 Rule-Based Strategy Selection

Each hotspot is assigned one or more management strategies drawn from a curated catalogue of internationally precedented parking interventions (for example, Taipei-style two-wheeler zoning, Bangkok’s Win System for auto-rickshaw management, Jakarta-style heavy-vehicle time-windows, SFpark/Tokyo-style high-impact tow-priority zones, and Vietnam-style Model Ward designations). Strategy selection is driven by the hotspot’s profile from Section 8 and its classification from Section 6: for example, a zone whose violations are dominated by two-wheelers above a threshold ratio triggers the two-wheeler zoning strategy as a candidate, with the specific threshold values configured centrally rather than scattered through the selection logic.

For each selected strategy, the system generates an implementation playbook: concrete, zone-specific guidance rather than a generic strategy description, to make the recommendation more directly actionable for a reviewer or, eventually, an enforcement planner.

### 9.2 Resource Allocation as a Linear Programming Problem

Consistent with the position in Section 3 that workforce allocation is fundamentally an optimisation problem rather than a prediction problem, we implemented a linear-programming-based allocator: a PCU-weighted objective function, with per-zone minimum officer bounds tied to whichever strategies were selected for that zone, subject to a total-officer capacity constraint. If the solver cannot satisfy all minimum bounds within capacity, the system falls back to a simpler proportional allocation rather than failing outright. This module is implemented and works correctly on its own; it is not yet fully wired into the live, request-driven dashboard path, which currently uses a simpler linear heuristic for responsiveness during the demo. We view connecting the two as a small remaining integration step rather than a research problem, since the

harder part, the solver itself, is already built and validated.

## 10 | Live Data Pipeline & Feedback Mechanism

### 10.1 Live Ingestion (Proof of Concept)

Stage (g) is implemented as a simulated live-feed endpoint that appends synthetic violation records and re-runs the data-processing pipeline, intended to demonstrate the ingestion mechanism rather than to represent a production-grade streaming architecture. We treat real-time visualisation and scheduled batch re-analysis as architecturally distinct concerns: incoming violations can be reflected in the dashboard immediately, while the heavier hotspot-detection, classification, and prediction stages do not need to re-run on every incoming record and are better scheduled on a fixed time quantum (for example, nightly) to preserve system responsiveness.

### 10.2 Feedback Mechanism: Current Scope and Future Direction

Stage (h) proved to be the hardest stage to define cleanly. A central tension we found is this: a successful patrol intervention destroys its own control group. Once the system is live and patrols are dispatched based on its predictions, there is no longer an undisturbed day to compare outcomes against. We considered and set aside a “compare against a similar historic day with no patrol” mechanism for this reason. It is more useful as a one-time, pre-deployment back-testing step than as an ongoing feedback loop, since it has nothing left to compare against once the system is actually in use.

The current implementation is a deliberately simple placeholder: a zone can be flagged as having received a mitigating action, which reduces its displayed severity in the dashboard for demonstration purposes. The direction we intend to pursue beyond this hackathon round is a vehicle-impact reweighting mechanism: starting from an equal-weight prior across vehicle types (that is, the PCU map in Section 5 treated as a starting assumption rather than a fixed truth), and continuously adjusting per-vehicle-type impact weights based on observed records of vehicle mix, staff deployed, and outcome as they accumulate through normal operation. This is a credit-assignment problem that does not require a special experimental setup or a control group, unlike the rejected approach above, and can be fed by ordinary day-to-day operational data.

## 11 | Model Evaluation Methodology

We treat honest evaluation as a first-class part of this submission, not an afterthought. This section documents the methodology we designed and applied to validate the predictive engine described in Section 7.

### 11.1 Chronological Train / Validation / Test Split

Calendar-similarity prediction is vulnerable to a specific evaluation trap. If test dates are scattered randomly across the full dataset, a test date’s true nearest calendar-neighbours are often still sitting in the training pool, sometimes literally the adjacent day, which inflates apparent ac-

curacy without reflecting genuine forecasting skill. To avoid this, we adopted a strictly chronological split: an initial block of historical days serves purely as a growing candidate pool, followed by a validation block used for tuning  $K$ , feature weights, and the confidence threshold, followed by a held-out test block touched only once, after all tuning decisions are frozen.

## 11.2 Walk-Forward (Rolling-Origin) Validation

Rather than evaluating against one fixed candidate pool, we adopted walk-forward validation. To predict day  $N$ , the candidate pool consists of all days strictly before day  $N$ ; once day  $N$ 's actual outcome is known, it is folded into the pool before predicting day  $N + 1$ . This is a faithful simulation of how the system would actually behave if deployed live, since at any point in time only the past is available to predict the future, rather than a single static snapshot.

## 11.3 Ground Truth Definition

A zone-day is labelled an actual hotspot if its PCU-weighted impact meets or exceeds that zone's own historical median, computed from the candidate pool available at that point in the walk-forward process (that is, the same expanding-window logic used for prediction itself, applied to labelling). We are aware this definition has a specific limitation: it labels a date as a hotspot relative to each zone's own typical activity, meaning a quiet zone slightly above its own historically low median and a severely congested zone slightly below its own historically high median are not distinguished from one another. We have flagged this as an open modelling decision, with alternatives including an absolute impact threshold or a top- $N$  capacity-based definition, to be settled deliberately rather than as a side effect of an unrelated change.

## 11.4 Metrics

We deliberately avoided plain accuracy as a primary metric: with a meaningful fraction of zone-days being genuinely non-hotspot, a model that simply predicts "nothing unusual anywhere" can score deceptively well on accuracy while providing no operational value. We use:

- **Precision:** of the zones flagged as predicted hotspots, what fraction were genuinely hotspots. This is the more operationally relevant metric given that patrol dispatch is a resource-constrained decision: precision tells us how often that dispatch would have been justified.
- **Recall:** of the zones that were genuinely hotspots, what fraction the model successfully flagged. Relevant if the cost of a missed hotspot (unpatrolled congestion) is judged higher than the cost of an unnecessary dispatch.
- **F1-Score:** the harmonic mean, used as a single-number summary while we do not yet have a validated, department-specific cost ratio between false positives and false negatives.

We did not pick a single "correct" priority between precision and recall, since that tradeoff is a policy decision about real-world enforcement costs that this team is not positioned to make on its own. We report both, along with a confidence-threshold sweep showing how each varies across the full range of thresholds, so the tradeoff is visible rather than hidden behind one chosen number.

## 11.5 Confidence Threshold Selection

The confidence threshold used to decide “is this zone a predicted hotspot” is selected empirically rather than guessed: we sweep the threshold across its full range on the validation set under the walk-forward methodology described above, compute precision, recall, and F1 at each point, and select the threshold that performs best against our chosen objective. That frozen threshold is then applied exactly once to the held-out test set, and the resulting metrics are the ones we report as the system’s final, honest performance.

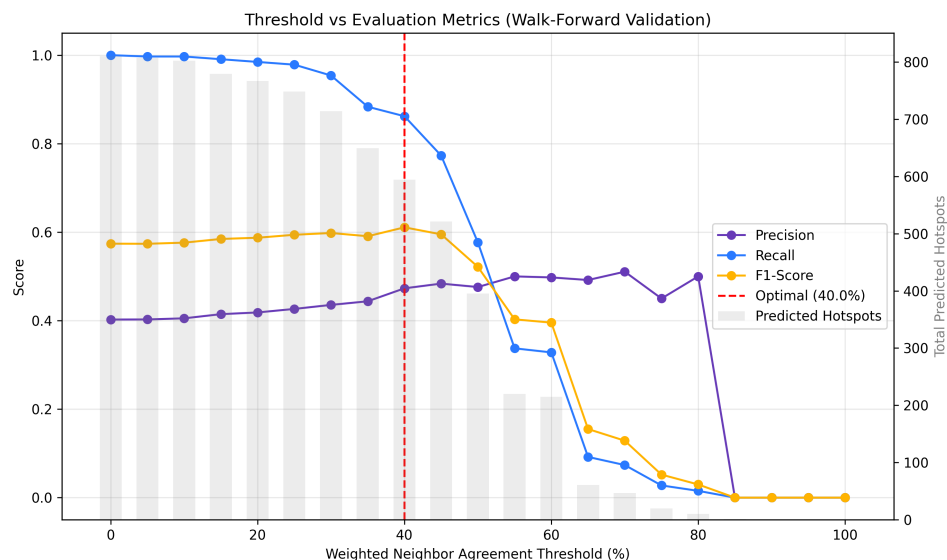


Figure 1: Precision, recall, and F1-score swept across the full confidence-threshold range under walk-forward validation. The grey bars show the total number of zones predicted as hotspots at each threshold (right axis). The selected optimal threshold of 40.0% sits at the point where F1 is highest, balancing the steep recall drop-off against the more gradual precision gain visible at higher thresholds. We label the x-axis *Weighted Neighbor Agreement Threshold* rather than “confidence” or “probability,” since the underlying score is a  $K$ -neighbour vote fraction and has not yet been statistically calibrated.

As Figure 1 shows, recall starts near 1.0 at a threshold of 0% (predicting almost every zone as a hotspot) and falls steadily as the threshold rises, while precision rises slowly from roughly 0.40 toward 0.45 to 0.50 over the same range, before both drop toward zero once the threshold exceeds about 85% and too few zones clear the bar to form a meaningful sample. F1 is highest at a threshold of 40.0%, which we adopted as the operating threshold for the test-set evaluation reported below.

## 11.6 Validation Results

Figure 2 reports confusion-matrix outcomes averaged across all 20 validation days, rather than a single representative day, addressing a methodological gap we found during internal review: a single day’s outcome can be misleading, since the steepness of the recall curve in Figure 1 means metrics can shift considerably between adjacent threshold values. Reporting the genuine mean across the full validation window, with the day-to-day averaging implemented in code rather than entered by hand, was a deliberate fix applied after our internal review found that the original version of this chart had not been computed this way.

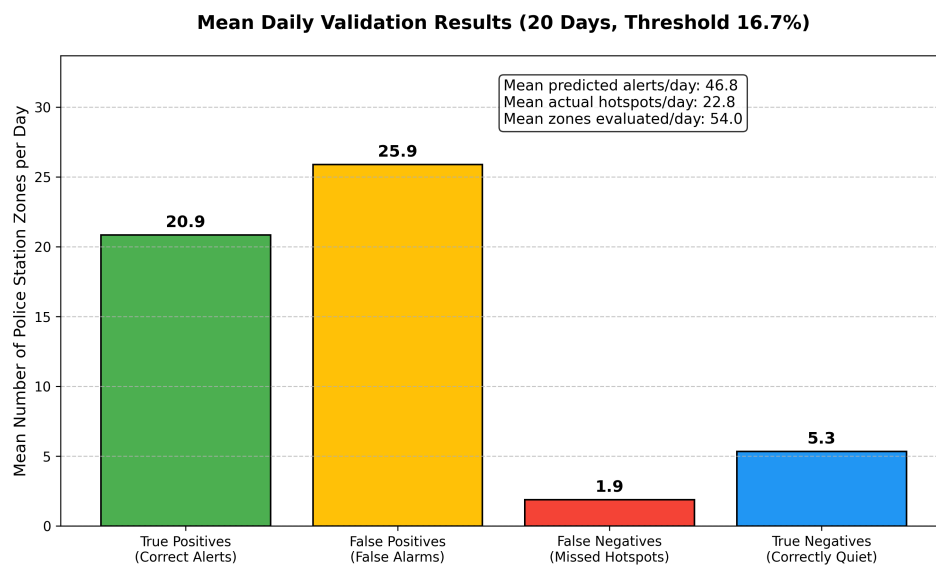


Figure 2: Mean daily confusion matrix across all 20 validation days, evaluated at a 16.7% threshold (one of the swept threshold points, distinct from the final 40.0% operating threshold, shown here to illustrate the averaging methodology). Each bar is the mean count, per day, of police-station zones falling into that outcome category, out of a mean of 54.0 zones evaluated per day.

## 12 | Dashboard & Frontend Implementation

The dashboard is a single-page application built with vanilla JavaScript, Leaflet (with the leaflet.heat plugin) for the interactive map, and Chart.js for time-pattern and breakdown visualisations, styled with a dark, control-room-inspired theme meant to evoke an operations and dispatch context rather than a generic analytics dashboard.

### 12.1 Layout

- A tabbed interface separating the Hotspot Map, Time Patterns, Violation & Vehicle Breakdown, and Zone Leaderboard views, so each analytical concern has dedicated screen space rather than being compressed into one dense view.
- A persistent filter bar (day of week, violation tag, vehicle type, hour range) that applies consistently across whichever view is active.
- A “Predict Tomorrow” toggle that switches the entire dashboard, including the map, stats, and leaderboard, between the live rolling 7-day historical view and the next-day prediction view, staying in whichever state the user has selected until switched back.
- A zone-detail side panel and modal showing the full decision-engine output, including selected strategies, reasoning, and implementation playbook, for any zone the user selects on the map or leaderboard.

## 13 | Future Scope

We see the following as the most valuable next steps for this project, roughly in the order we would pursue them:

- Acquire and integrate an independent traffic-flow or travel-time dataset (for example, road-speed or GPS-probe data) for the road segments inside our highest-ranked hotspots, and correlate violation density against measured slowdowns. This is, in our assessment, the single most important next step: it would let the system answer the original problem brief's "impact on traffic flow" question directly and empirically, rather than relying on PCU weighting as a proxy for impact.
- Resolve the open ground-truth definition question (Section 11) deliberately, likely in consultation with whoever would operationally use this system's output, since the right definition depends on how the output would actually be acted on.
- Complete the integration of the linear-programming resource allocator into the live dashboard path, replacing the current simplified heuristic.
- Build out patrol route and timing optimisation (stage f) as a proper constrained Operations Research problem, incorporating real shift-length, break, and travel-time constraints once available.
- Develop the vehicle-impact reweighting feedback mechanism described in Section 10 into a working system, fed by real deployment outcomes rather than the current placeholder mitigation flag.
- Replace the simulated live-feed endpoint with a genuine streaming ingestion pipeline, while keeping the architectural separation between real-time visualisation and scheduled batch re-analysis established in Section 10.
- Pursue formal calibration of the confidence score (for example, via isotonic regression or Platt scaling against the walk-forward validation outcomes) so that a stated confidence value can be presented as a genuinely calibrated probability rather than a raw neighbour-agreement fraction.
- As more historical data accumulates over time, revisit the calendar-similarity model's handling of rare events, major festivals in particular, since the current five-month dataset window limits how much can be learned about them.

## Appendix A: Technology Stack Summary

| Layer                     | Technology                                                          |
|---------------------------|---------------------------------------------------------------------|
| Data processing           | Python, pandas, numpy                                               |
| Spatial clustering        | Custom grid-based density clustering (DBSCAN-equivalent)            |
| Predictive modelling      | Custom weighted-Euclidean $k$ -Nearest-Neighbours                   |
| Feature weight derivation | scikit-learn (Random Forest, Linear Regression, Mutual Information) |
| Resource allocation       | Linear programming solver                                           |
| Backend / API             | Python (HTTP server, JSON API endpoints)                            |
| Frontend mapping          | Leaflet.js + leaflet.heat                                           |
| Frontend charting         | Chart.js                                                            |
| Frontend styling          | Custom CSS, dark control-room theme                                 |